[10/29/2026 12:36 AM] Nakama :

ESHRAQ UNIVERSITY

Department of Computer Engineering

Embedding Systems

A Comprehensive Study of the Longest Remaining Time First (LRTF) CPU Scheduling Algorithm:

Mathematical Foundations, Scheduling Behavior, Performance Evaluation, and Comparative Analysis

STUDENT NAME

Zoha Karimifar

INSTRUCTOR

Majid Shamshirgaran

Submission Date: 21 October 2026

Abstract

This paper presents a comprehensive academic analysis of the Longest Remaining Time First (LRTF) CPU scheduling algorithm. LRTF is a preemptive scheduling discipline that dynamically assigns the processor to the ready process possessing the largest remaining execution requirement. While conceptually simple, LRTF exhibits complex behaviors regarding fairness, response times, and context-switch overhead, making it a vital subject of study in operating systems and embedded computing. We formalize the LRTF selection rule mathematically, provide a detailed step-by-step algorithmic flow, and demonstrate its behavior through rigorous worked examples. The paper explores the implementation methodology, contrasting simplified educational simulators with robust synchronization mechanisms such as semaphores and monitors. Furthermore, we conduct a structured comparison against widely adopted algorithms, including First Come First Served (FCFS), Shortest Remaining Time First (SRTF), Earliest Deadline First (EDF), and Rate Monotonic (RM). The analysis critically evaluates the advantages and severe limitations of LRTF—particularly its tendency to induce starvation among short tasks—and outlines its relevance within embedded and real-time systems. This work serves as a complete pedagogical resource and an empirical framework for evaluating execution-time-driven scheduling policies.

Keywords: LRTF, CPU Scheduling, Operating Systems, Preemptive Scheduling, EDF, Rate Monotonic, Embedded Systems.

[10/29/2026 12:36 AM] Nakama :

Table of Contents

1. Introduction

2. Fundamentals of CPU Scheduling

3. Definition and Principles of LRTF

4. Mathematical Foundations

5. LRTF Scheduling Procedure and Flowchart

6. Worked Scheduling Examples

7. Implementation Methodology

8. Performance Evaluation

9. Comparison with Other Scheduling Algorithms

10. Advantages, Limitations, and Failure Scenarios

11. LRTF in Embedded and Real-Time Systems

12. Synchronization Considerations

13. Discussion and Critical Analysis

14. Final Conclusion and Future Work

References

1. Introduction

The central nervous system of any modern operating system is its CPU scheduler. In a multitasking environment, multiple processes compete for the finite resource of processor time. The scheduling algorithm determines which process runs, when it runs, and for how long. The design of this algorithm profoundly impacts system throughput, latency, fairness, and overall user experience [1].

Scheduling algorithms are broadly categorized into non-preemptive and preemptive disciplines. In non-preemptive scheduling, once a process is allocated the CPU, it retains control until it terminates or voluntarily yields. Conversely, preemptive scheduling empowers the operating system to forcibly interrupt a running task to allocate the processor to a higher-priority task [2].

Among preemptive algorithms, execution-time-driven policies like the Shortest Remaining Time First (SRTF) aim to minimize average waiting time. The **Longest Remaining Time First (LRTF)** algorithm operates on the exact opposite premise: at any scheduling point, the processor is assigned to the ready process with the largest remaining execution time.

Research Problem and Significance: While LRTF is often dismissed in practical system design due to its poor average-case performance, its study is critically important. It illustrates the consequences of ignoring deadlines and periods, serving as an excellent theoretical baseline. Furthermore, in specific batch-processing or specialized load-balancing scenarios, LRTF-like policies can be useful for ensuring uniform completion times across a set of tasks.

Methodology and Scope: This paper operates within the framework of uniprocessor, preemptive scheduling with independent

tasks. We will define the formal mathematical model, trace execution through deterministic examples, and evaluate its performance against standard alternatives like EDF and RM.

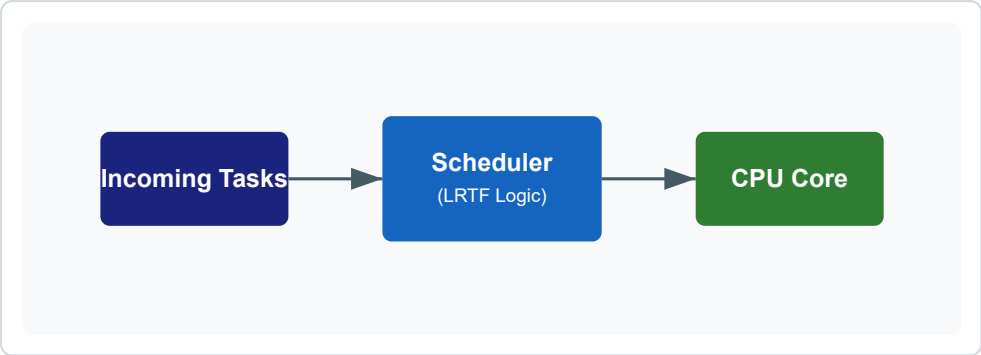


Figure 1: *General architecture of a CPU scheduler within an operating system.*

2. Fundamentals of CPU Scheduling

To understand LRTF, one must first master the fundamental terminology of process scheduling. A process transitions through several states: *New*, *Ready*, *Running*, *Waiting*, and *Terminated*. The scheduler selects tasks exclusively from the *Ready* queue.

Table 1: Core CPU Scheduling Terminology

Term	Definition
Arrival Time (A_i)	The time at which a process enters the ready queue.
Burst Time (C_i)	The total CPU time required by the process to complete execution.
Remaining Time ($r_i(t)$)	The amount of CPU time still needed by the process at time t.
Completion Time (F_i)	The time at which the process finishes execution.
Turnaround Time (TAT)	The total time from arrival to completion ($F_i - A_i$).
Waiting Time (WT)	The total time spent waiting in the ready queue (TAT - Burst Time).
Response Time (RT)	The time from arrival to the first allocation of the CPU.
Context Switch	The overhead of saving the state of one process and loading another.

[10/29/2026 12:36 AM] Nakama :

Scheduling can be non-preemptive (the running task cannot be interrupted) or preemptive (the OS can interrupt the running task).

LRTF is strictly preemptive, adding significant complexity compared to non-preemptive algorithms.

Table 2: Preemptive vs. Non-Preemptive Scheduling

Feature	Non-Preemptive	Preemptive (e.g., LRTF)
Interruption	Not allowed	Allowed at scheduling events
Overhead	Low (few context switches)	High (frequent context switches)
Responsiveness	Poor for interactive tasks	Can be better for high-priority tasks
Implementation Complexity	Simple	Complex (requires interrupt handling)

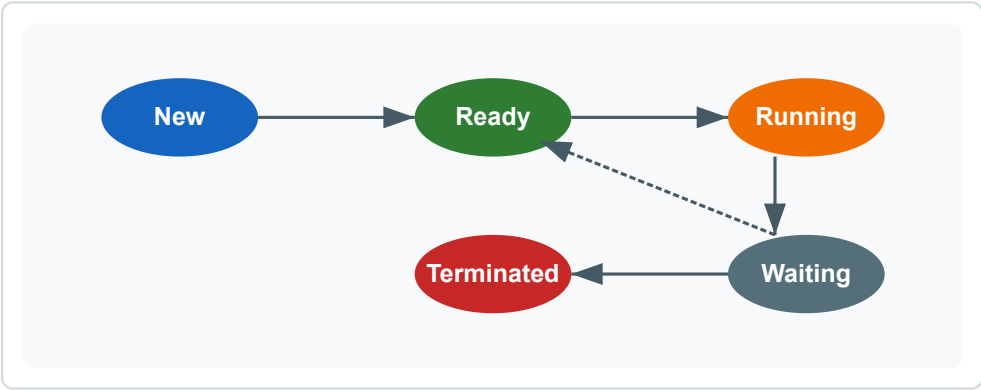


Figure 2: *Process state transitions during scheduling.*

3. Definition and Principles of LRTF

The Longest Remaining Time First (LRTF) algorithm is the preemptive variant of the Longest Job First (LJF) algorithm. At any scheduling decision point, the scheduler evaluates all processes currently in the ready queue and selects the one with the maximum remaining execution time. [10/29/2026 12:36 AM] Nakama :

$$p(t) = \arg\max_{i \in R(t)} r_i(t) \tag{3.1}$$

Where $R(t)$ is the set of ready, unfinished processes at time t, and $r_i(t)$ is the remaining execution time of process i.

$$r_i(t) = \max\left(0, C_i - e_i(t)\right) \tag{3.2}$$

Where C_i is the total required CPU burst and $e_i(t)$ is the accumulated CPU execution received by process i up to time t.

3.1. The Role of Preemption and Events

Unlike continuous theoretical models, real LRTF implementations do not constantly evaluate priorities at every nanosecond. Instead, scheduling decisions occur at discrete **scheduling events**:

- When a new process arrives (enters the ready queue).
- When a running process completes.
- When a running process blocks (e.g., waiting for I/O).
- At a periodic timer interrupt (time slice expiration).

Preemption occurs if, at a scheduling event, a newly arrived process has a larger remaining time than the currently running process.

3.2. Tie-Breaking Rules

If multiple processes have the exact same maximum remaining time, the algorithm must specify a tie-breaking rule. Standard rules include:

- **First Come, First Served (FCFS):** The process that arrived earliest wins.
- **Lowest PID:** The process with the lowest numerical identifier wins.

The choice of tie-breaking rule can significantly alter the Gantt chart and final performance metrics, making it essential for reproducibility.

Table 3: LRTF Mathematical Notation and Variables

Symbol	Description
A_i	Arrival time of process i
C_i	Total burst time required by process i
$r_i(t)$	Remaining time of process i at time t
S_i	Start time of process i
F_i	Completion time of process i
D_i	Absolute deadline (if applicable)
$R(t)$	Set of ready processes at time t

4. Mathematical Foundations

Evaluating the performance of LRTF requires rigorous mathematical metrics. The fundamental performance indicators are turnaround time, waiting time, and response time.

$$T_i^{\text{turn}} = F_i - A_i \tag{4.1}$$

Turnaround time represents the total lifespan of the process in the system. Waiting time is the portion of time spent strictly in the ready queue:

[10/29/2026 12:36 AM] Nakama :

$$W_i = T_i^{\text{turn}} - C_i \tag{4.2}$$

Response time measures the delay before the user/system sees initial progress:

$$R_i = S_i - A_i \tag{4.3}$$

The average metrics for a set of n processes are calculated as follows:

$$\overline{T}^{\text{turn}} = \frac{1}{n}\sum_{i=1}^{n} T_i^{\text{turn}} \quad \text{and} \quad \overline{W} = \frac{1}{n}\sum_{i=1}^{n} W_i \tag{4.4}$$

CPU utilization and throughput are system-wide metrics:

$$U_{\text{CPU}} = \frac{\text{CPU busy time}}{\text{total observation time}} \times 100\% \tag{4.5}$$

$$\text{Throughput} = \frac{\text{number of completed processes}}{\text{observation interval}} \tag{4.6}$$

If context-switch overhead (O_{cs}) is modeled, the effective execution time of a process increases by the number of switches it incurs. Thus,

actual CPU utilization degrades proportionally to the frequency of preemptions induced by the LRTF policy.

Table 4: Performance Metrics and Their Mathematical Definitions

Metric	Formula	Optimization Goal
Turnaround Time	$F_i - A_i$	Minimize
Waiting Time	$T_i^{\text{turn}} - C_i$	Minimize
Response Time	$S_i - A_i$	Minimize
CPU Utilization	$\frac{\text{Busy}}{\text{Total}} \times 100\%$	Maximize
Throughput	$\frac{N_{\text{completed}}}{\Delta t}$	Maximize

5. LRTF Scheduling Procedure and Flowchart

The following flowchart illustrates the complete logic of an LRTF scheduler, including handling of idle periods, arrivals, preemption, and completion.

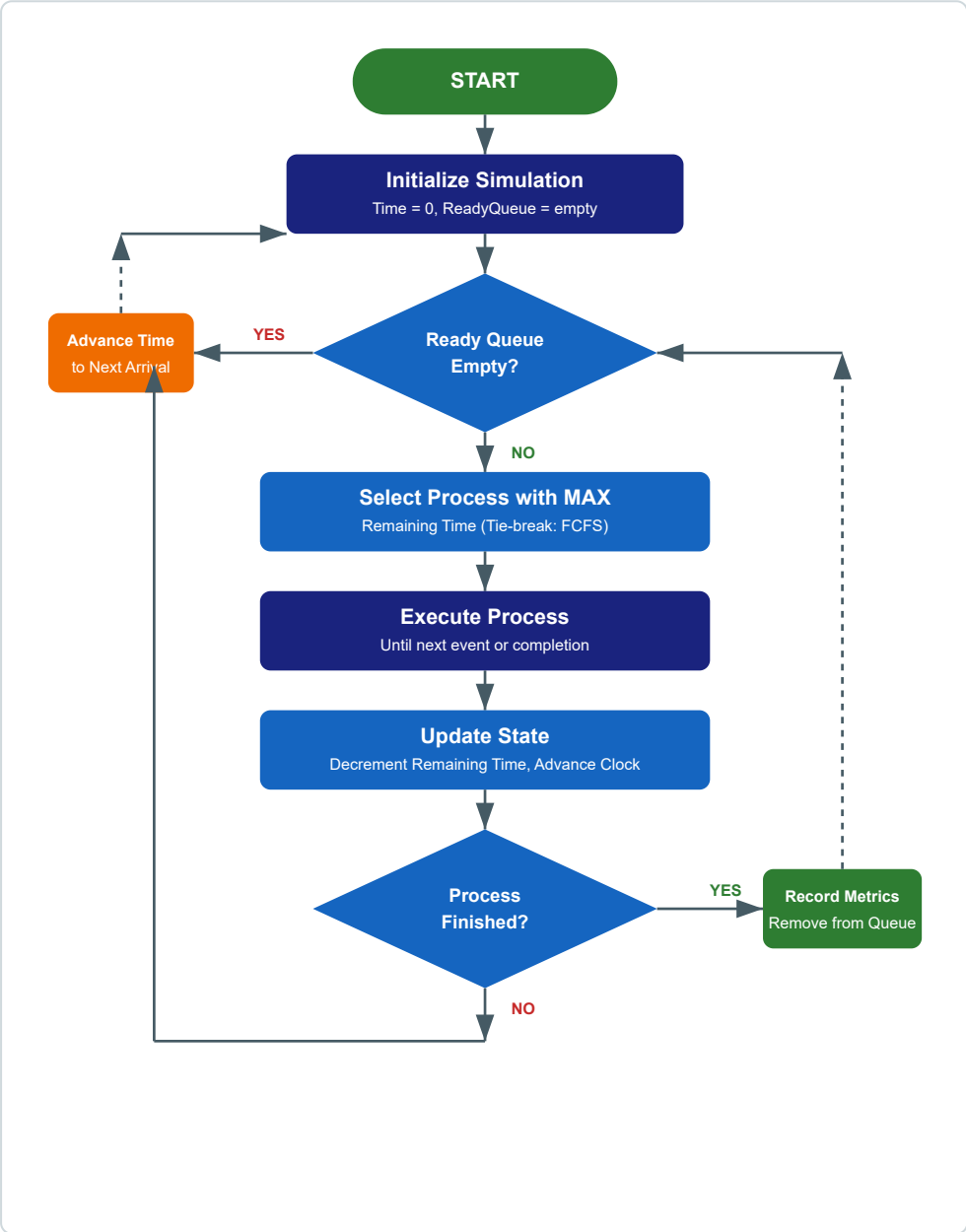


Figure 3: *Complete flowchart of the LRTF scheduling algorithm.*

5.1. Pseudocode for LRTF

```
function LRTF_Scheduler(ProcessList P):
    currentTime = 0
    completed = 0
    n = length(P)

    while completed < n:
        # 1. Add arrivals to ready queue
        for process in P:
            if process.arrivalTime == currentTime:
                ReadyQueue.enqueue(process)

        # 2. Handle idle CPU
        if ReadyQueue.isEmpty():
            currentTime++
            continue

        # 3. Select process with max remaining time
        # (Assuming tie-break by lowest arrival time, then PID)
        selected = null
        maxRemaining = -1

        for process in ReadyQueue:
            if process.remainingTime > maxRemaining:
                maxRemaining = process.remainingTime
                selected = process
            else if process.remainingTime == maxRemaining:
                if process.arrivalTime < selected.arrivalTime:
                    selected = process

        # 4. Execute selected process for 1 time unit
        selected.remainingTime--
        currentTime++

        # 5. Check for completion
        if selected.remainingTime == 0:
            selected.completionTime = currentTime
            ReadyQueue.remove(selected)
            completed++
```

```
    return calculateMetrics(P)
```

6. Worked Scheduling Examples

Example 1: Basic LRTF Scheduling

Consider the following set of three processes. The tie-breaking rule is strictly **Earliest Arrival Time, then Lowest PID**.

Table 5: Example 1 Input Processes

Process	Arrival Time ( )	Burst Time ( )
P1	0	8
P2	1	4
P3	2	6

[10/29/2026 12:36 AM] Nakama :

Detailed Trace:

- **t=0:** P1 arrives. Ready: {P1(8)}. P1 runs.
- **t=1:** P2 arrives. Ready: {P1(7), P2(4)}. Max is P1(7). P1 runs.
- **t=2:** P3 arrives. Ready: {P1(6), P2(4), P3(6)}. Tie between P1 and P3. P1 arrived earlier. P1 runs.
- **t=3:** Ready: {P1(5), P2(4), P3(6)}. Max is P3(6). P3 preempts P1. P3 runs.
- **t=4:** Ready: {P1(5), P2(4), P3(5)}. Tie P1/P3. P1 wins (arrived earlier). P1 runs.
- **t=5:** Ready: {P1(4), P2(4), P3(5)}. Max P3(5). P3 runs.
- **t=6:** Ready: {P1(4), P2(4), P3(4)}. Tie P1/P2/P3. P1 wins. P1 runs.

- **t=7:** Ready: {P1(3), P2(4), P3(4)}. Tie P2/P3. P2 arrived earlier. P2 runs.
- **t=8:** Ready: {P1(3), P2(3), P3(4)}. Max P3(4). P3 runs.
- **t=9:** Ready: {P1(3), P2(3), P3(3)}. Tie P1/P2/P3. P1 wins. P1 runs.
- **t=10:** Ready: {P1(2), P2(3), P3(3)}. Tie P2/P3. P2 runs.
- **t=11:** Ready: {P1(2), P2(2), P3(3)}. Max P3(3). P3 runs.
- **t=12:** Ready: {P1(2), P2(2), P3(2)}. Tie P1/P2/P3. P1 wins. P1 runs.
- **t=13:** Ready: {P1(1), P2(2), P3(2)}. Tie P2/P3. P2 runs.
- **t=14:** Ready: {P1(1), P2(1), P3(2)}. Max P3(2). P3 runs.
- **t=15:** Ready: {P1(1), P2(1), P3(1)}. Tie P1/P2/P3. P1 wins. P1 runs.
- **t=16:** P1 finishes. Ready: {P2(1), P3(1)}. Tie. P2 wins. P2 runs.
- **t=17:** P2 finishes. Ready: {P3(1)}. P3 runs.
- **t=18:** P3 finishes.



Figure 4: *Gantt chart for Example 1 illustrating complex preemption patterns.*

Table 6: Example 1 Final Scheduling Metrics

Process	Arrival	Burst	Completion	Turnaround	Waiting
P1	0	8	16	16	8
P2	1	4	17	16	12
P3	2	6	18	16	10
Average				**16.0**	**10.0**

Interpretation: Notice that all three processes complete at approximately the same time (16, 17, 18). This is a hallmark of LRTF. However, the average waiting time (10.0) is significantly worse than what SJF or SRTF would achieve for the same workload.

[10/29/2026 12:41 AM] Nakama :

7. Implementation Methodology

Implementing an LRTF scheduler for an operating system or a simulation requires a robust architecture. The primary components include the Process Control Block (PCB) array, the ready queue, and the timer/event handler.

Table 7: Simulation Architecture and Modules

Module	Responsibility
Task Generator	Creates processes with specified arrival and burst times.
Ready Queue Manager	Maintains active processes. Handles insertion on arrival and removal on completion.
Priority Evaluator	Scans the ready queue to find the maximum remaining time.
Event Handler	Triggers scheduling decisions upon arrivals, completions, or timer ticks.
Metrics Collector	Records TAT, WT, RT, and context switch counts.

A critical design choice is whether to simulate time continuously or event-by-event. Event-driven simulation is mathematically equivalent but computationally faster.

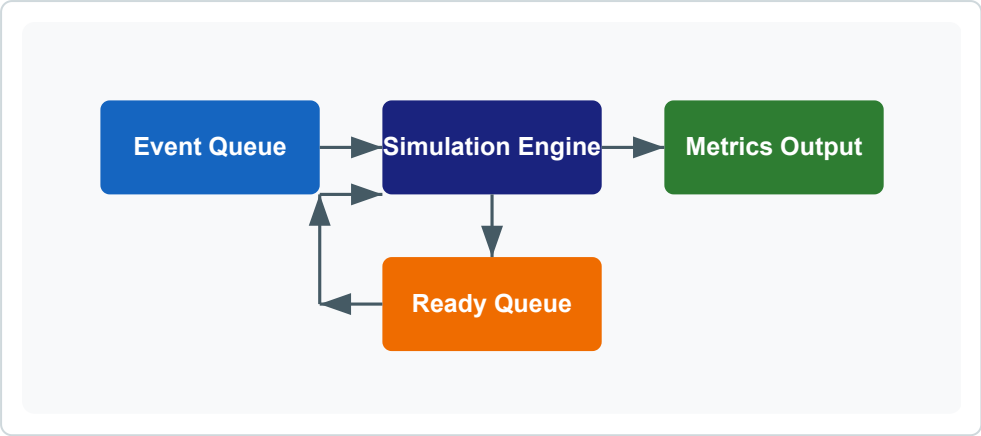


Figure 5: *Architecture of an event-driven LRTF simulation engine.*

[10/29/2026 12:41 AM] Nakama :

8. Performance Evaluation

To evaluate LRTF effectively, we define controlled workloads. In real-time systems, the metric of success is often deadline satisfaction, whereas in general-purpose systems, it is average waiting time or response time.

Table 8: Evaluation Scenarios and Workload Assumptions

Scenario	Arrival Pattern	Burst Time Dist.	Primary Metric
Batch Processing	All at t=0	Highly varied	Average Completion Time
Interactive System	Poisson	Short (I/O bound)	Response Time
Embedded Control	Periodic	Deterministic	Deadline Miss Rate

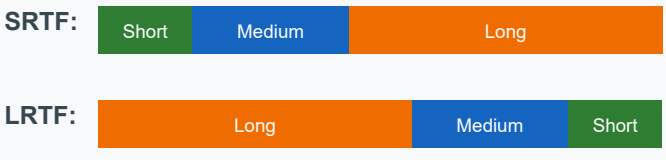
9. Comparison with Other Scheduling Algorithms

LRTF is rarely used in isolation. Its behavior becomes clear when contrasted with established algorithms.

Table 9: Comprehensive Comparison of Scheduling Algorithms

Algorithm	Priority Rule	Preemptive?	Starvation Risk	Objective
FCFS	Arrival Time	No	No	Simplicity
SJF	Shortest Burst	No	Yes (Long jobs)	Min Avg Waiting Time
SRTF	Shortest Remaining	Yes	Yes (Long jobs)	Min Avg Waiting Time
Round Robin	FCFS with Time Quantum	Yes	No	Fairness / Response
LRTF	Longest Remaining	Yes	Yes (Short jobs)	Simultaneous Completion
EDF	Earliest Deadline	Yes	Possible (Overload)	Real-time Feasibility
RM	Shortest Period	Yes	No	Predictable Real-time

LRTF vs SRTF: While SRTF minimizes average waiting time by favoring short tasks, LRTF maximizes it by favoring long tasks. LRTF is the theoretical worst-case scenario for average waiting time.



Comparison of execution order for one Short, one Medium, and one Long task.

Figure 6: *Conceptual comparison of task ordering: SRTF vs LRTF.*

10. Advantages, Limitations, and Failure Scenarios

10.1. Advantages

- **Theoretical Value:** LRTF is a fundamental concept for understanding execution-time driven scheduling.
- **Simultaneous Completion:** For a finite set of simultaneously arriving tasks, LRTF ensures they all finish at roughly the same time.

10.2. Limitations

- **Starvation:** Short tasks are constantly preempted by long tasks. If new long tasks keep arriving, short tasks will never execute.
- **Terrible Average Waiting Time:** It systematically maximizes the waiting time of the majority of tasks.
- **Context Switch Overhead:** The aggressive preemptive nature leads to frequent context switches, degrading CPU utilization.

10.3. Failure Scenario: The Starvation Trap

Imagine a short task P_short(10ms) arrives at t=1. A long task P_long1(100ms) arrives at t=2. P_short runs for 1ms, then P_long1 preempts it. If at t=10, P_long2(200ms) arrives, it preempts P_long1. P_short is pushed further and further back, potentially never completing.

11. LRTF in Embedded and Real-Time Systems

In embedded systems, timing predictability is paramount. Tasks often have strict deadlines tied to physical events (e.g., reading a sensor every 10ms).

LRTF ignores deadlines entirely. A task with an imminent deadline but a small remaining execution time will be delayed by a task with a distant deadline and a large execution time. Therefore, **LRTF is fundamentally unsuitable for hard real-time systems**.

Table 10: Embedded-System Timing Constraints

Constraint	Requirement	LRTF Compatibility
Determinism	Predictable execution time	Low (frequent preemptions)
Deadline Adherence	Strict $F_i \leq D_i$	None (ignores D_i)
Low Overhead	Minimal context switches	Poor
Jitter	Minimal variation in start time	High

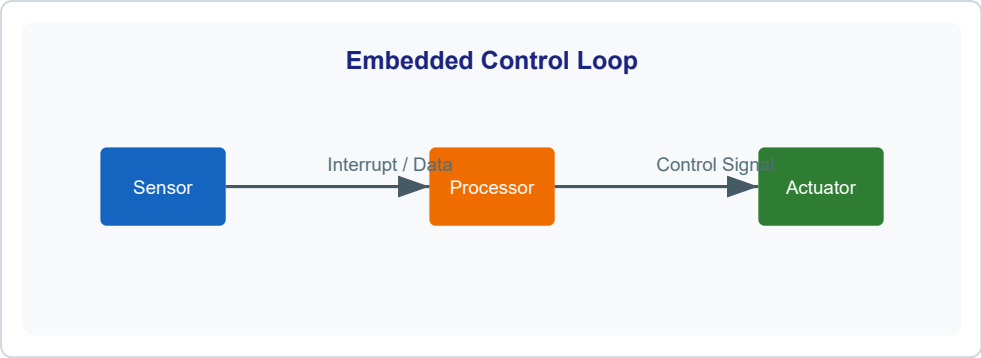


Figure 7: *Typical embedded system control loop where timing constraints are critical.*

12. Synchronization Considerations

A common point of confusion is the relationship between LRTF and synchronization primitives like semaphores and monitors.

LRTF is a scheduling policy. It dictates *which* process runs next. **Semaphores and monitors are synchronization mechanisms.** They dictate *how* processes safely share resources.

Implementing an LRTF scheduler requires synchronization to protect the ready queue from concurrent access by interrupt handlers and the scheduler thread. A typical implementation uses a mutex (binary semaphore) to lock the queue.

- **Semaphores:** Good for low-level, highly efficient kernel locking. Risk of priority inversion if a high-priority LRTF task blocks on a mutex held by a low-priority task.
- **Monitors:** Higher-level abstraction. A monitor can encapsulate the ready queue and condition variables, ensuring only one thread modifies it at a time, reducing race conditions.

Using a semaphore or monitor to protect the ready queue does *not* change the LRTF priority rule; it merely makes the implementation thread-safe.

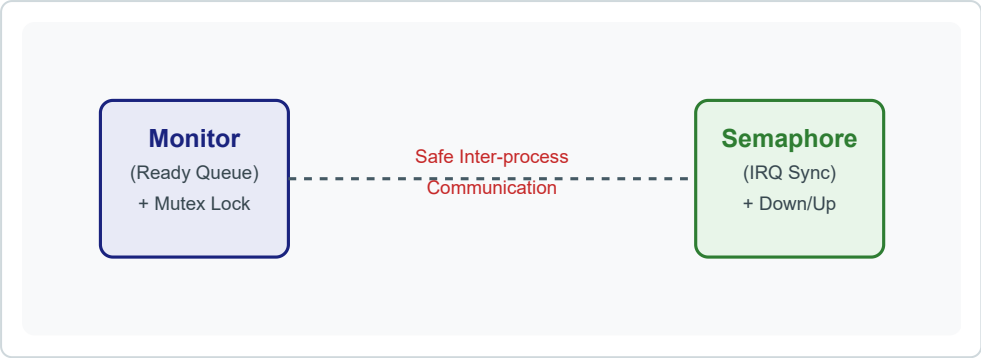


Figure 8: *Synchronization mechanisms used to protect scheduling data structures.*

13. Discussion and Critical Analysis

The Longest Remaining Time First algorithm serves as a vital pedagogical tool. Its primary optimization goal is equalizing the completion times of all processes in a batch. However, it directly antagonizes the goal of minimizing average waiting time or response time.

A critical flaw of LRTF is its lack of deadline awareness. In modern computing—especially in embedded environments—tasks are not just competing for CPU time; they are bound by physical constraints. Consequently, EDF (Earliest Deadline First) and RM (Rate Monotonic) are vastly superior for real-time systems because they factor in the temporal constraints of the tasks.

When context-switch overhead is factored into the theoretical models, LRTF performs even worse. The frequent preemptions inherent in prioritizing the longest task result in severe CPU utilization degradation.

14. Final Conclusion and Future Work

In conclusion, the Longest Remaining Time First algorithm is mathematically elegant but practically inefficient for general-purpose or real-time operating systems. It successfully guarantees that no task is left behind in a static workload by forcing all tasks to finish simultaneously, but it does so at the expense of crippling the responsiveness of short tasks.

Future Work: While standard LRTF is impractical, variants could be explored. For example, a hybrid scheduler could use LRTF for background batch processing while reserving SRTF or EDF for foreground tasks. Future research could also investigate the impact of

dynamic tie-breaking rules on LRTF's starvation behavior, or apply machine learning to predict when an LRTF-like policy would be beneficial in a cloud computing cluster.

References

1. A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Wiley, 2018.
2. A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Pearson, 2015. [10/29/2026 12:41 AM] Nakama :
3. W. Stallings, *Operating Systems: Internals and Design Principles*, 9th ed. Pearson, 2018.
4. C. L. Liu and J. W. Layland, "Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973.
5. G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3rd ed. Springer, 2011.
6. GeeksforGeeks, "Longest Remaining Time First (LRTF) or Preemptive Longest Job First CPU Scheduling Algorithm," 2026. [Online]. Available: https://www.geeksforgeeks.org